

操作系统实验文档

Lab 3 - Bare Metal

537000a9

Built by github-actions, at 2025-03-04 14:43:45+08:00

1. Weekly Lab Material

1.1 Bare Metal Programming

When programming in a Linux environment, the `libc` functions and well-encapsulated system calls provided by the Linux environment allow us to focus primarily on the logic of our program to interact with the operating system. For instance, when we use `printf` and `scanf` to operate on standard input/output streams, the user-space runtime environment is provided by `libc`, while the kernel environment is supplied by the Linux Kernel.

However, when writing our own operating system, we lack the runtime environment provided by Linux or any other operating system. Instead, we interact directly with the CPU and hardware. Such programs, which directly interface with the CPU and hardware, are called **bare-metal programs**.

What is a Bare-Metal Program

A **bare-metal program** is a type of software that runs directly on the hardware of a device without relying on an underlying operating system (OS). Essentially, it's code that interacts with the hardware at the most fundamental level, controlling the processor, memory, input/output (I/O) devices, and other components directly.

In our xv6 experiments, we will write operating system programs on the RISC-V architecture.

Lab Step 1: Running the First Bare-Metal Program

xv6-lab1 Code Branch

<https://github.com/yuk1i/SUSTech-OS-2025/tree/xv6-lab1>

Use the command `git clone https://github.com/yuk1i/SUSTech-OS-2025 -b xv6-lab1 xv6lab1` to download the xv6-lab1 code.

It is recommended to download the code for each lab in a separate location.


```

Platform IPI Device      : aclint-mswi
Platform Timer Device    : aclint-mtimer @ 100000000Hz
Platform Console Device  : uart8250
Platform HSM Device      : ---
Platform PMU Device      : ---
Platform Reboot Device   : syscon-reboot
Platform Shutdown Device : syscon-poweroff
Platform Suspend Device  : ---
Platform CPPC Device     : ---
Firmware Base           : 0x80000000
Firmware Size           : 327 KB
Firmware RW Offset      : 0x40000
Firmware RW Size        : 71 KB
Firmware Heap Offset    : 0x49000
Firmware Heap Size      : 35 KB (total), 2 KB (reserved), 11 KB (used), 21 KB (free)
Firmware Scratch Size   : 4096 B (total), 416 B (used), 3680 B (free)
Runtime SBI Version     : 2.0

```

```

Domain0 Name           : root
Domain0 Boot HART      : 0
Domain0 HARTs          : 0*
Domain0 Region00       : 0x0000000000100000-0x0000000000100fff M: (I,R,W) S/U: (R,W)
Domain0 Region01       : 0x0000000001000000-0x0000000001000fff M: (I,R,W) S/U: (R,W)
Domain0 Region02       : 0x0000000002000000-0x000000000200ffff M: (I,R,W) S/U: ( )
Domain0 Region03       : 0x0000000008004000-0x0000000008005fff M: (R,W) S/U: ( )
Domain0 Region04       : 0x0000000008000000-0x0000000008003fff M: (R,X) S/U: ( )
Domain0 Region05       : 0x000000000c400000-0x000000000c5fffff M: (I,R,W) S/U: (R,W)
Domain0 Region06       : 0x000000000c000000-0x000000000c3fffff M: (I,R,W) S/U: (R,W)
Domain0 Region07       : 0x0000000000000000-0xffffffffffffffff M: ( ) S/U: (R,W,X)
Domain0 Next Address   : 0x0000000080200000
Domain0 Next Arg1      : 0x000000009fe00000
Domain0 Next Mode      : S-mode
Domain0 SysReset       : yes
Domain0 SysSuspend     : yes

```

```

Boot HART ID           : 0
Boot HART Domain       : root
Boot HART Priv Version  : v1.12
Boot HART Base ISA     : rv64imafdc
Boot HART ISA Extensions : sstc,zicntr,zihpm,zicboz,zicbom,sdtrig
Boot HART PMP Count     : 16
Boot HART PMP Granularity : 2 bits
Boot HART PMP Address Bits: 54
Boot HART MHPM Info     : 16 (0x0007fff8)
Boot HART Debug Triggers : 2 triggers
Boot HART MIDELEG       : 0x0000000000001666
Boot HART MEDELEG       : 0x0000000000f0b509

```

```
clean bss: 0x0000000080207000 - 0x0000000080207000
```

```
Kernel booted.
```

```
Hello World!
```

```
sysregs:
```

```

sstatus : 0x80000000200006000
scause  : 0x0000000000000000
sepc    : 0x0000000000000000
stval   : 0x0000000000000000
sip     : 0x0000000000000000
sie     : 0x0000000000000000
satp    : 0x0000000000000000

```

```
0x00000000deadbeef
```

```
kernel ends, parking...
```

i How to Exit QEMU

Press `ctrl` and `A` simultaneously, release them, then press `x`.

If this method doesn't work due to prior key presses, you can directly close the terminal.

Lab Step 1 Completed

The above lab process demonstrates the boot process of a minimal kernel. Next, we will explain how an operating system starts running from the perspective of **privilege levels**. First, let's understand what privilege levels are.

1.1.1 Privilege Levels

i Privilege Levels (riscv-privileged.pdf)

Privilege levels is a concept in computer systems used to define the access control and permissions that different processes or users have within the system.

The existence of privilege levels aims to protect system resources, ensure security, and isolate processes based on trust and functionality.

At any time, a RISC-V hardware thread (hart) is running at some privilege level encoded as a mode in one or more CSRs (control and status registers).

Three RISC-V privilege levels are currently defined as shown in Table 1.1.

Privilege levels are used to provide protection between different components of the software stack, and attempts to perform operations not permitted by the current privilege mode will cause an exception to be raised.

These exceptions will normally cause traps into an underlying execution environment.

Privilege levels are a critical state of the CPU during execution, indicating the level of privilege the currently running code possesses. Code with the highest privilege level has unrestricted access to all resources, such as all physical memory and peripherals. Higher privilege levels can restrict the resources accessible to lower privilege levels. For example, a lower privilege level cannot access memory or CSR registers owned by a higher privilege level. However, when the CPU is running at a lower privilege level, the program can actively or passively switch to a higher privilege level and execute predefined code.

i Why Privilege Levels Are Needed

The primary purpose of introducing privilege levels in CPU design is to ensure system security and stability. In modern computer systems, the operating system needs to strictly manage and control hardware resources. Privilege levels help the operating system isolate the kernel from user programs, preventing untrusted code or applications from improperly manipulating critical system components.

For example, CPU time is an important "hardware resource." Modern operating systems achieve the illusion that multiple user programs are running simultaneously by scheduling them to execute in turns. The operating system (running at a high privilege level) manages CPU time by forcing an application to pause after executing for a certain period, switching to the next application.

Without privilege levels to enforce this, relying on applications to voluntarily yield CPU time would allow a malicious program to monopolize the CPU, causing all other applications on the system to become unresponsive.

Privilege levels are **implemented in the CPU hardware circuitry**, not simulated through software. In RISC-V, privilege levels are distinguished using 2 bits, defining three modes: **M mode**, **S mode**, and **U mode**.

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	<i>Reserved</i>	
3	11	Machine	M

Table 1.1: RISC-V privilege levels.

The machine level has the highest privileges and is the only mandatory privilege level for a RISC-V hardware platform. Code run in machine-mode (M-mode) is usually inherently trusted, as it has low-level access to the machine implementation. M-mode can be used to manage secure execution environments on RISC-V. User-mode (U-mode) and supervisor-mode (S-mode) are intended for conventional application and operating system usage respectively.

- **M mode:** The highest privilege level in the RISC-V architecture, typically running firmware like OpenSBI, with direct access to physical memory.
- **S mode:** Designed for operating systems, capable of setting up virtual memory.
- **U mode:** Designed for user programs, with the least privileges, also supporting virtual memory.

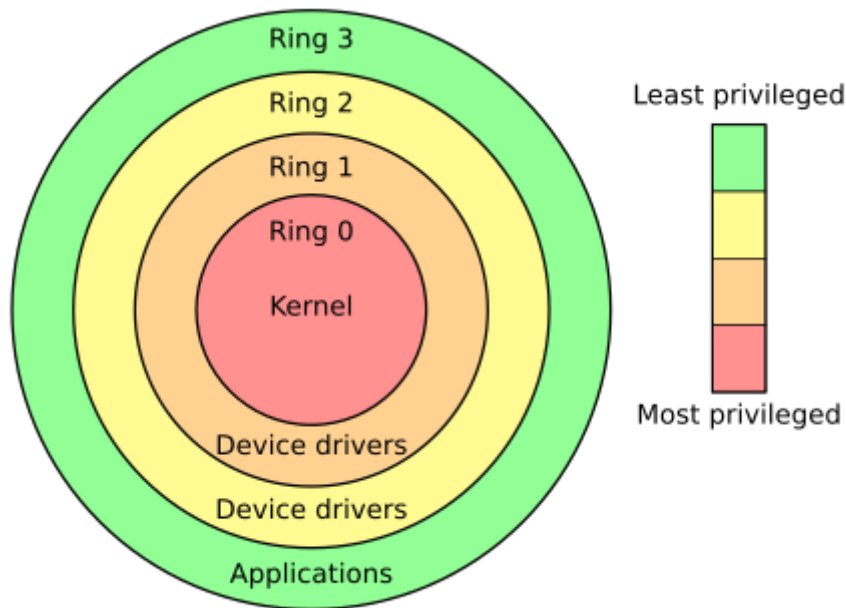
In bare-metal programming, we need to initialize the CPU state, including basic functionalities like page tables and interrupts. These CPU states are controlled via **CSRs (Control and Status Registers)**, which are generally only accessible and modifiable by higher privilege levels.

AArch64 & x86

Modern instruction set architectures all define different privilege levels.

The **AArch64** (ARM64) architecture, commonly used in mobile devices, defines four privilege levels (Exception Levels) from lowest to highest: **EL0**, **EL1**, **EL2**, and **EL3**. Similar to RISC-V, the highest privilege level, **EL3**, runs low-level firmware (Secure Monitor), **EL1** runs the operating system (OS), **EL0** runs user programs, and **EL2** runs virtual machine programs (Hypervisor).

The **x86** (IA32 & AMD64) architecture defines privilege levels as four **Rings**: **Ring 0** is the highest privilege level, running the operating system, while **Ring 3** is the lowest, running user programs. Typically, x86 systems only use **Ring 0** and **Ring 3**.



1.1.2 CSR (Control and Status Registers)

Unlike the 32 general-purpose registers, CSRs are special registers.

CSR (Control and Status Registers) are critical register sets used to control CPU behavior, store system state, and manage exceptions and interrupts. CSRs require special instructions to access.

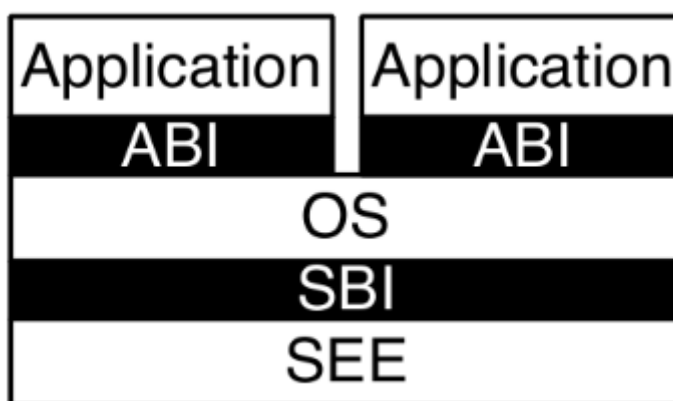
Each CSR has privilege-level restrictions. For example: The `time` and `cycle` registers can be read by **U mode** but cannot be modified. S-mode CSRs like `sstatus` can be read and written by **S mode** and **M mode**. M-mode CSRs like `mstatus` can only be read and written by **M mode**. Attempting to access a higher-privilege CSR from a lower privilege level will trigger an exception.

Each CSR contains defined fields that occupy specific bits, representing particular meanings. We'll explore CSRs further in subsequent lessons.

1.1.3 Runtime Environment

Across the three privilege levels in RISC-V, the architecture defines three runtime states: **Firmware (Machine mode)**, **Operating System (Supervisor mode)**, and **User Mode (User mode)**.

In RISC-V, the runtime environment provided by the operating system (Supervisor) to applications is called the **ABI (Application Binary Interface)**, while the runtime environment provided by the firmware (Machine mode, SEE) to the operating system (Supervisor, OS) is called the **SBI (Supervisor Binary Interface)**.



OpenSBI provides access interfaces to basic hardware and offers services to **S mode** through a syscall-like mechanism known as **SBI Calls**. These include basic serial I/O functions like `sbi_console_putchar` and `sbi_console_getchar`.

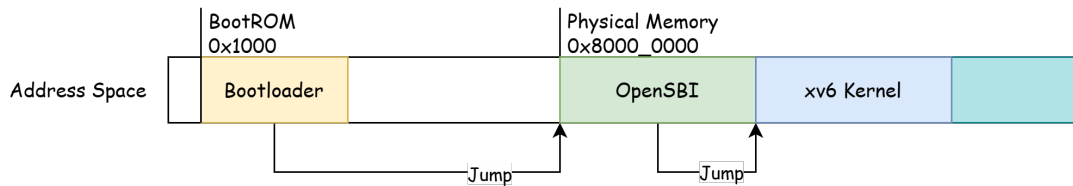
1.1.4 Boot Process Overview

The boot process of an operating system involves initializing each level of the runtime environment. Typically, we start initializing from the highest privilege level and progressively descend to lower privilege levels.

From the moment the CPU powers on and resets, it begins executing the first instruction. At this point, the CPU typically executes code from a small ROM area, either internal or external to the chip, known as the **Bootloader**. The bootloader can be considered "Stage 0" of the boot process, with its primary tasks being to locate the next stage's image, copy it into memory, and jump to it.

On the RISC-V platform, the next stage is the initialization of the **SBI (Supervisor Binary Interface)**, which we implement using **OpenSBI**. Thus, in Stage 0, the

OpenSBI image and our kernel image are loaded into memory. The bootloader then jumps the CPU to the OpenSBI load address, starting its execution and entering the next stage.



In this stage, OpenSBI initializes critical system devices. Once OpenSBI completes the **M mode** initialization, it downgrades the CPU privilege to **S mode** and jumps to the kernel's entry point, officially starting kernel code execution.

Below, we'll explain the tasks of each stage in detail:

Stage 0: From CPU Power-On

Upon powering on, the CPU undergoes a hardware-level reset. Recall how, in digital logic courses, we reset a `reg` variable when `rst_n` is triggered.

The reset sets the **Program Counter (PC)** to a fixed value called the **Reset Vector**, the address of the first instruction executed after power-on. This address typically points to a **Block ROM** on the RISC-V chip, a read-only code region. After executing the Reset Vector code, control flow jumps to the next stage. In QEMU, this next stage is the **M mode** OpenSBI.

What is Implementation-Defined Behavior?

The Reset Vector value for each RISC-V hardware is determined by its implementation. The RISC-V manual does not mandate a specific Reset Vector value, making it **Implementation-Defined**.

Lab Step 2: GDB Debugging to Observe the Boot Process

GDB Debugging

GDB (GNU Debugger) is a powerful open-source debugging tool, primarily used for debugging programs written in languages like C and C++. It helps developers identify and fix errors by supporting features such as setting breakpoints, stepping through code, inspecting variable values, and checking memory states.

Using GDB, we can set breakpoints and observe the operating system's execution process while running it in QEMU.

First, add the line `set auto-load safe-path /` to your `~/.gdbinit` file to allow GDB to automatically load the `.gdbinit` file in the current directory.

In one terminal, run `make debug`. This launches `qemu-system-riscv64` with the parameters `-S -gdb tcp::3333`, indicating that it will wait for a debugger to attach.

Then, in another terminal (ensuring the working directory matches the first terminal), run `gdb-multiarch`. This starts the GDB debugger and automatically loads the `.gdbinit` file in the current directory.

If everything works correctly, GDB will pause at address `0x1000`, the Reset Vector address for the QEMU platform.

```

laptop@laptop /d/o/SUSTech-OS-2025 (xv6-lab1)> make debug
qemu-system-riscv64 -nographic -machine virt -cpu rv64,svadu=off -m 512 -kernel build/kernel -S
DB tcp::3333

There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
GDB for linux ready, type "quit" to start, "quit config" to configure
88 commands loaded and 5 functions added for GDB 15.2 in 0.00ms using Python engine 3.12
The target architecture is set to "riscv:rv64".
(qemu) (gdb) in ?? ()
[ Legend: Modified register | Code | Heap | Stack | String ]

registers
$zero: 0x0000000000000000 $ra: 0x0000000000000000 $sp: 0x0000000000000000
$fp: 0x0000000000000000 $lr: 0x0000000000000000 $t0: 0x0000000000000000
$t1: 0x0000000000000000 $t2: 0x0000000000000000 $t3: 0x0000000000000000
$t4: 0x0000000000000000 $t5: 0x0000000000000000 $t6: 0x0000000000000000
$t7: 0x0000000000000000 $t8: 0x0000000000000000 $t9: 0x0000000000000000
$t10: 0x0000000000000000 $t11: 0x0000000000000000 $t12: 0x0000000000000000
$t13: 0x0000000000000000 $t14: 0x0000000000000000 $t15: 0x0000000000000000
$t16: 0x0000000000000000 $t17: 0x0000000000000000 $t18: 0x0000000000000000
$t19: 0x0000000000000000 $t20: 0x0000000000000000 $t21: 0x0000000000000000
$t22: 0x0000000000000000 $t23: 0x0000000000000000 $t24: 0x0000000000000000
$t25: 0x0000000000000000 $t26: 0x0000000000000000 $t27: 0x0000000000000000
$t28: 0x0000000000000000 $t29: 0x0000000000000000 $t30: 0x0000000000000000
$t31: 0x0000000000000000 $t32: 0x0000000000000000 $t33: 0x0000000000000000
$t34: 0x0000000000000000 $t35: 0x0000000000000000 $t36: 0x0000000000000000
$t37: 0x0000000000000000 $t38: 0x0000000000000000 $t39: 0x0000000000000000
$t40: 0x0000000000000000 $t41: 0x0000000000000000 $t42: 0x0000000000000000
$t43: 0x0000000000000000 $t44: 0x0000000000000000 $t45: 0x0000000000000000
$t46: 0x0000000000000000 $t47: 0x0000000000000000 $t48: 0x0000000000000000
$t49: 0x0000000000000000 $t50: 0x0000000000000000 $t51: 0x0000000000000000
$t52: 0x0000000000000000 $t53: 0x0000000000000000 $t54: 0x0000000000000000
$t55: 0x0000000000000000 $t56: 0x0000000000000000 $t57: 0x0000000000000000
$t58: 0x0000000000000000 $t59: 0x0000000000000000 $t60: 0x0000000000000000
$t61: 0x0000000000000000 $t62: 0x0000000000000000 $t63: 0x0000000000000000
$t64: 0x0000000000000000 $t65: 0x0000000000000000 $t66: 0x0000000000000000
$t67: 0x0000000000000000 $t68: 0x0000000000000000 $t69: 0x0000000000000000
$t70: 0x0000000000000000 $t71: 0x0000000000000000 $t72: 0x0000000000000000
$t73: 0x0000000000000000 $t74: 0x0000000000000000 $t75: 0x0000000000000000
$t76: 0x0000000000000000 $t77: 0x0000000000000000 $t78: 0x0000000000000000
$t79: 0x0000000000000000 $t80: 0x0000000000000000 $t81: 0x0000000000000000
$t82: 0x0000000000000000 $t83: 0x0000000000000000 $t84: 0x0000000000000000
$t85: 0x0000000000000000 $t86: 0x0000000000000000 $t87: 0x0000000000000000
$t88: 0x0000000000000000 $t89: 0x0000000000000000 $t90: 0x0000000000000000
$t91: 0x0000000000000000 $t92: 0x0000000000000000 $t93: 0x0000000000000000
$t94: 0x0000000000000000 $t95: 0x0000000000000000 $t96: 0x0000000000000000
$t97: 0x0000000000000000 $t98: 0x0000000000000000 $t99: 0x0000000000000000
$t100: 0x0000000000000000 $t101: 0x0000000000000000 $t102: 0x0000000000000000
$t103: 0x0000000000000000 $t104: 0x0000000000000000 $t105: 0x0000000000000000
$t106: 0x0000000000000000 $t107: 0x0000000000000000 $t108: 0x0000000000000000
$t109: 0x0000000000000000 $t110: 0x0000000000000000 $t111: 0x0000000000000000
$t112: 0x0000000000000000 $t113: 0x0000000000000000 $t114: 0x0000000000000000
$t115: 0x0000000000000000 $t116: 0x0000000000000000 $t117: 0x0000000000000000
$t118: 0x0000000000000000 $t119: 0x0000000000000000 $t120: 0x0000000000000000
$t121: 0x0000000000000000 $t122: 0x0000000000000000 $t123: 0x0000000000000000
$t124: 0x0000000000000000 $t125: 0x0000000000000000 $t126: 0x0000000000000000
$t127: 0x0000000000000000 $t128: 0x0000000000000000 $t129: 0x0000000000000000
$t130: 0x0000000000000000 $t131: 0x0000000000000000 $t132: 0x0000000000000000
$t133: 0x0000000000000000 $t134: 0x0000000000000000 $t135: 0x0000000000000000
$t136: 0x0000000000000000 $t137: 0x0000000000000000 $t138: 0x0000000000000000
$t139: 0x0000000000000000 $t140: 0x0000000000000000 $t141: 0x0000000000000000
$t142: 0x0000000000000000 $t143: 0x0000000000000000 $t144: 0x0000000000000000
$t145: 0x0000000000000000 $t146: 0x0000000000000000 $t147: 0x0000000000000000
$t148: 0x0000000000000000 $t149: 0x0000000000000000 $t150: 0x0000000000000000
$t151: 0x0000000000000000 $t152: 0x0000000000000000 $t153: 0x0000000000000000
$t154: 0x0000000000000000 $t155: 0x0000000000000000 $t156: 0x0000000000000000
$t157: 0x0000000000000000 $t158: 0x0000000000000000 $t159: 0x0000000000000000
$t160: 0x0000000000000000 $t161: 0x0000000000000000 $t162: 0x0000000000000000
$t163: 0x0000000000000000 $t164: 0x0000000000000000 $t165: 0x0000000000000000
$t166: 0x0000000000000000 $t167: 0x0000000000000000 $t168: 0x0000000000000000
$t169: 0x0000000000000000 $t170: 0x0000000000000000 $t171: 0x0000000000000000
$t172: 0x0000000000000000 $t173: 0x0000000000000000 $t174: 0x0000000000000000
$t175: 0x0000000000000000 $t176: 0x0000000000000000 $t177: 0x0000000000000000
$t178: 0x0000000000000000 $t179: 0x0000000000000000 $t180: 0x0000000000000000
$t181: 0x0000000000000000 $t182: 0x0000000000000000 $t183: 0x0000000000000000
$t184: 0x0000000000000000 $t185: 0x0000000000000000 $t186: 0x0000000000000000
$t187: 0x0000000000000000 $t188: 0x0000000000000000 $t189: 0x0000000000000000
$t190: 0x0000000000000000 $t191: 0x0000000000000000 $t192: 0x0000000000000000
$t193: 0x0000000000000000 $t194: 0x0000000000000000 $t195: 0x0000000000000000
$t196: 0x0000000000000000 $t197: 0x0000000000000000 $t198: 0x0000000000000000
$t199: 0x0000000000000000 $t200: 0x0000000000000000 $t201: 0x0000000000000000
$t202: 0x0000000000000000 $t203: 0x0000000000000000 $t204: 0x0000000000000000
$t205: 0x0000000000000000 $t206: 0x0000000000000000 $t207: 0x0000000000000000
$t208: 0x0000000000000000 $t209: 0x0000000000000000 $t210: 0x0000000000000000
$t211: 0x0000000000000000 $t212: 0x0000000000000000 $t213: 0x0000000000000000
$t214: 0x0000000000000000 $t215: 0x0000000000000000 $t216: 0x0000000000000000
$t217: 0x0000000000000000 $t218: 0x0000000000000000 $t219: 0x0000000000000000
$t220: 0x0000000000000000 $t221: 0x0000000000000000 $t222: 0x0000000000000000
$t223: 0x0000000000000000 $t224: 0x0000000000000000 $t225: 0x0000000000000000
$t226: 0x0000000000000000 $t227: 0x0000000000000000 $t228: 0x0000000000000000
$t229: 0x0000000000000000 $t230: 0x0000000000000000 $t231: 0x0000000000000000
$t232: 0x0000000000000000 $t233: 0x0000000000000000 $t234: 0x0000000000000000
$t235: 0x0000000000000000 $t236: 0x0000000000000000 $t237: 0x0000000000000000
$t238: 0x0000000000000000 $t239: 0x0000000000000000 $t240: 0x0000000000000000
$t241: 0x0000000000000000 $t242: 0x0000000000000000 $t243: 0x0000000000000000
$t244: 0x0000000000000000 $t245: 0x0000000000000000 $t246: 0x0000000000000000
$t247: 0x0000000000000000 $t248: 0x0000000000000000 $t249: 0x0000000000000000
$t250: 0x0000000000000000 $t251: 0x0000000000000000 $t252: 0x0000000000000000
$t253: 0x0000000000000000 $t254: 0x0000000000000000 $t255: 0x0000000000000000
$t256: 0x0000000000000000 $t257: 0x0000000000000000 $t258: 0x0000000000000000
$t259: 0x0000000000000000 $t260: 0x0000000000000000 $t261: 0x0000000000000000
$t262: 0x0000000000000000 $t263: 0x0000000000000000 $t264: 0x0000000000000000
$t265: 0x0000000000000000 $t266: 0x0000000000000000 $t267: 0x0000000000000000
$t268: 0x0000000000000000 $t269: 0x0000000000000000 $t270: 0x0000000000000000
$t271: 0x0000000000000000 $t272: 0x0000000000000000 $t273: 0x0000000000000000
$t274: 0x0000000000000000 $t275: 0x0000000000000000 $t276: 0x0000000000000000
$t277: 0x0000000000000000 $t278: 0x0000000000000000 $t279: 0x0000000000000000
$t280: 0x0000000000000000 $t281: 0x0000000000000000 $t282: 0x0000000000000000
$t283: 0x0000000000000000 $t284: 0x0000000000000000 $t285: 0x0000000000000000
$t286: 0x0000000000000000 $t287: 0x0000000000000000 $t288: 0x0000000000000000
$t289: 0x0000000000000000 $t290: 0x0000000000000000 $t291: 0x0000000000000000
$t292: 0x0000000000000000 $t293: 0x0000000000000000 $t294: 0x0000000000000000
$t295: 0x0000000000000000 $t296: 0x0000000000000000 $t297: 0x0000000000000000
$t298: 0x0000000000000000 $t299: 0x0000000000000000 $t300: 0x0000000000000000
$t301: 0x0000000000000000 $t302: 0x0000000000000000 $t303: 0x0000000000000000
$t304: 0x0000000000000000 $t305: 0x0000000000000000 $t306: 0x0000000000000000
$t307: 0x0000000000000000 $t308: 0x0000000000000000 $t309: 0x0000000000000000
$t310: 0x0000000000000000 $t311: 0x0000000000000000 $t312: 0x0000000000000000
$t313: 0x0000000000000000 $t314: 0x0000000000000000 $t315: 0x0000000000000000
$t316: 0x0000000000000000 $t317: 0x0000000000000000 $t318: 0x0000000000000000
$t319: 0x0000000000000000 $t320: 0x0000000000000000 $t321: 0x0000000000000000
$t322: 0x0000000000000000 $t323: 0x0000000000000000 $t324: 0x0000000000000000
$t325: 0x0000000000000000 $t326: 0x0000000000000000 $t327: 0x0000000000000000
$t328: 0x0000000000000000 $t329: 0x0000000000000000 $t330: 0x0000000000000000
$t331: 0x0000000000000000 $t332: 0x0000000000000000 $t333: 0x0000000000000000
$t334: 0x0000000000000000 $t335: 0x0000000000000000 $t336: 0x0000000000000000
$t337: 0x0000000000000000 $t338: 0x0000000000000000 $t339: 0x0000000000000000
$t340: 0x0000000000000000 $t341: 0x0000000000000000 $t342: 0x0000000000000000
$t343: 0x0000000000000000 $t344: 0x0000000000000000 $t345: 0x0000000000000000
$t346: 0x0000000000000000 $t347: 0x0000000000000000 $t348: 0x0000000000000000
$t349: 0x0000000000000000 $t350: 0x0000000000000000 $t351: 0x0000000000000000
$t352: 0x0000000000000000 $t353: 0x0000000000000000 $t354: 0x0000000000000000
$t355: 0x0000000000000000 $t356: 0x0000000000000000 $t357: 0x0000000000000000
$t358: 0x0000000000000000 $t359: 0x0000000000000000 $t360: 0x0000000000000000
$t361: 0x0000000000000000 $t362: 0x0000000000000000 $t363: 0x0000000000000000
$t364: 0x0000000000000000 $t365: 0x0000000000000000 $t366: 0x0000000000000000
$t367: 0x0000000000000000 $t368: 0x0000000000000000 $t369: 0x0000000000000000
$t370: 0x0000000000000000 $t371: 0x0000000000000000 $t372: 0x0000000000000000
$t373: 0x0000000000000000 $t374: 0x0000000000000000 $t375: 0x0000000000000000
$t376: 0x0000000000000000 $t377: 0x0000000000000000 $t378: 0x0000000000000000
$t379: 0x0000000000000000 $t380: 0x0000000000000000 $t381: 0x0000000000000000
$t382: 0x0000000000000000 $t383: 0x0000000000000000 $t384: 0x0000000000000000
$t385: 0x0000000000000000 $t386: 0x0000000000000000 $t387: 0x0000000000000000
$t388: 0x0000000000000000 $t389: 0x0000000000000000 $t390: 0x0000000000000000
$t391: 0x0000000000000000 $t392: 0x0000000000000000 $t393: 0x0000000000000000
$t394: 0x0000000000000000 $t395: 0x0000000000000000 $t396: 0x0000000000000000
$t397: 0x0000000000000000 $t398: 0x0000000000000000 $t399: 0x0000000000000000
$t400: 0x0000000000000000 $t401: 0x0000000000000000 $t402: 0x0000000000000000
$t403: 0x0000000000000000 $t404: 0x0000000000000000 $t405: 0x0000000000000000
$t406: 0x0000000000000000 $t407: 0x0000000000000000 $t408: 0x0000000000000000
$t409: 0x0000000000000000 $t410: 0x0000000000000000 $t411: 0x0000000000000000
$t412: 0x0000000000000000 $t413: 0x0000000000000000 $t414: 0x0000000000000000
$t415: 0x0000000000000000 $t416: 0x0000000000000000 $t417: 0x0000000000000000
$t418: 0x0000000000000000 $t419: 0x0000000000000000 $t420: 0x0000000000000000
$t421: 0x0000000000000000 $t422: 0x0000000000000000 $t423: 0x0000000000000000
$t424: 0x0000000000000000 $t425: 0x0000000000000000 $t426: 0x0000000000000000
$t427: 0x0000000000000000 $t428: 0x0000000000000000 $t429: 0x0000000000000000
$t430: 0x0000000000000000 $t431: 0x0000000000000000 $t432: 0x0000000000000000
$t433: 0x0000000000000000 $t434: 0x0000000000000000 $t435: 0x0000000000000000
$t436: 0x0000000000000000 $t437: 0x0000000000000000 $t438: 0x0000000000000000
$t439: 0x0000000000000000 $t440: 0x0000000000000000 $t441: 0x0000000000000000
$t442: 0x0000000000000000 $t443: 0x0000000000000000 $t444: 0x0000000000000000
$t445: 0x0000000000000000 $t446: 0x0000000000000000 $t447: 0x0000000000000000
$t448: 0x0000000000000000 $t449: 0x0000000000000000 $t450: 0x0000000000000000
$t451: 0x0000000000000000 $t452: 0x0000000000000000 $t453: 0x0000000000000000
$t454: 0x0000000000000000 $t455: 0x0000000000000000 $t456: 0x0000000000000000
$t457: 0x0000000000000000 $t458: 0x0000000000000000 $t459: 0x0000000000000000
$t460: 0x0000000
```

? Lab Report 2

Determine what content QEMU loads into memory during startup.

Referring to Lab Report 1, open a GDB terminal and run the command `monitor info roms`. Fill the output information into the table in your lab report.

In each output line, `addr` indicates the starting address of the loaded content, `size` indicates the length of the content, and `mem=rom/ram` indicates whether the segment is read-only ROM or writable RAM.

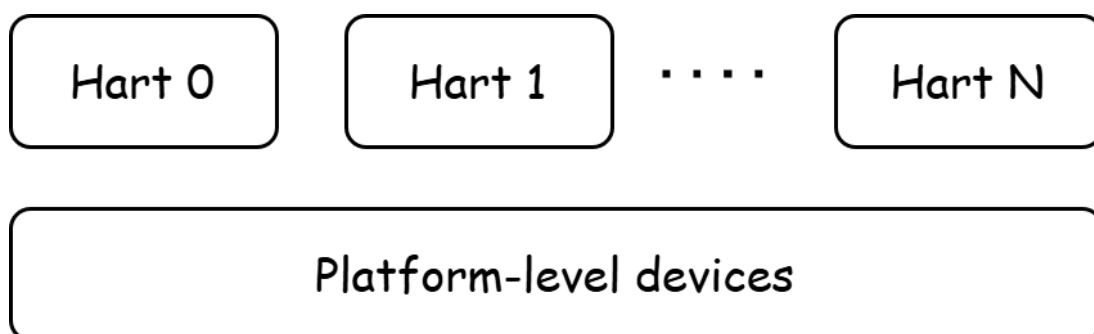
Lab Step 2 Completed

Next Stage: OpenSBI

In computers, **firmware** is a specific type of software that provides low-level control for a device's hardware and can load further software. For complex software (like an operating system), firmware provides a standardized operating environment. For simpler devices, firmware may act as the complete operating system, handling all control, monitoring, and data manipulation functions.

In x86-based systems, the firmware is **BIOS** or **UEFI**; in RISC-V-based systems, it is **OpenSBI**. OpenSBI runs in **M mode** because firmware requires direct hardware access.

At this stage, OpenSBI initializes critical system devices, referred to as **platform-level devices**. These are shared across the system, meaning multiple CPUs (or HARTs) see the same device.



```

Platform Name       : riscv-virtio,qemu
Platform Features   : medeleg
Platform HART Count : 1
Platform IPI Device  : aclint-mswi
Platform Timer Device : aclint-mtimer @ 100000000Hz
Platform Console Device : uart8250
Platform HSM Device  : ---
Platform PMU Device  : ---
Platform Reboot Device : syscon-reboot
  
```

```
Platform Shutdown Device : syscon-poweroff
Platform Suspend Device  : ---
```

Next, OpenSBI designates the next stage as our kernel:

```
Domain0 Next Address : 0x0000000080200000
Domain0 Next Arg1    : 0x000000009fe00000
Domain0 Next Mode    : S-mode
```

It also initializes HART-related configurations:

```
Boot HART ID      : 0
Boot HART Domain  : root
Boot HART Priv Version : v1.12
Boot HART Base ISA : rv64imafdc
```

Following Stage: Kernel Boot

After OpenSBI completes initialization, it downgrades to **S mode** and sets the PC to our kernel's starting address, `0x80200000`. This address holds the first instruction of the kernel's entry point, `_entry`, marking the point where CPU control is handed over to our xv6 kernel.

The `_entry` code is located in the `entry.S` file. But **how does this code end up at the kernel's starting address, `0x80200000`?**

COMPILATION PROCESS

Let's examine the kernel compilation process:

```
$ /d/o/SUSTech-OS-2025 (xv6-lab1)> make
riscv64-unknown-elf-gcc -fPIE -fno-pic -fno-plt -Wall -Wno-unused-variable -Werror -O -fno-omit-frame-
pointer -ggdb -march=rv64g -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -Ios -
std=gnu17 -fno-stack-protector -D LOG_LEVEL_ERROR -c os/console.c -o build/os/console.o
riscv64-unknown-elf-gcc -fPIE -fno-pic -fno-plt -Wall -Wno-unused-variable -Werror -O -fno-omit-frame-
pointer -ggdb -march=rv64g -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -Ios -
std=gnu17 -fno-stack-protector -D LOG_LEVEL_ERROR -c os/main.c -o build/os/main.o
riscv64-unknown-elf-gcc -fPIE -fno-pic -fno-plt -Wall -Wno-unused-variable -Werror -O -fno-omit-frame-
pointer -ggdb -march=rv64g -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -Ios -
std=gnu17 -fno-stack-protector -D LOG_LEVEL_ERROR -c os/printf.c -o build/os/printf.o
riscv64-unknown-elf-gcc -fPIE -fno-pic -fno-plt -Wall -Wno-unused-variable -Werror -O -fno-omit-frame-
pointer -ggdb -march=rv64g -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -Ios -
std=gnu17 -fno-stack-protector -D LOG_LEVEL_ERROR -c os/sbi.c -o build/os/sbi.o
riscv64-unknown-elf-gcc -fPIE -fno-pic -fno-plt -Wall -Wno-unused-variable -Werror -O -fno-omit-frame-
pointer -ggdb -march=rv64g -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -Ios -
std=gnu17 -fno-stack-protector -D LOG_LEVEL_ERROR -c os/string.c -o build/os/string.o
riscv64-unknown-elf-gcc -fPIE -fno-pic -fno-plt -Wall -Wno-unused-variable -Werror -O -fno-omit-frame-
pointer -ggdb -march=rv64g -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -Ios -
std=gnu17 -fno-stack-protector -D LOG_LEVEL_ERROR -c os/entry.S -o build/os/entry.o
riscv64-unknown-elf-ld -z max-page-size=4096 -T os/kernel.ld -o build/kernel build/os/console.o build/os/
main.o build/os/printf.o build/os/sbi.o build/os/string.o build/os/entry.o
riscv64-unknown-elf-objdump -S build/kernel > build/kernel.asm
```

```
riscv64-unknown-elf-objdump -t build/kernel | sed '1,/SYMBOL TABLE/d; s/ .* //; /^$/d' > build/kernel.sym
Build kernel done
```

Let's break down the compilation parameters:

`gcc` is the familiar C compiler, while `riscv64-unknown-elf-gcc` specifies a GCC suite for RISC-V 64-bit, targeting an unknown platform and producing ELF-format output.

Key compilation flags:

- `-march=rv64g -mmodel=medany -mno-relax :`

Targets the `rv64g` architecture, uses the `medany` addressing model, and disables linker relaxation.

See: <https://gcc.gnu.org/onlinedocs/gcc/RISC-V-Options.html>

- `-ffreestanding -fno-common -nostdlib :`

Indicates no reliance on standard library functions and avoids assuming standard definitions for common functions (e.g., `memset`).

- `-fno-pie -no-pie -fno-plt -fno-omit-frame-pointer -fno-stack-protector :`

This indicates that the generated ELF file should not use Position-Independent Executable (PIE) code, as it would result in the creation of the Global Offset Table (GOT) and Procedure Linkage Table (PLT) sections. Our kernel is currently unable to handle such complex structures.

`-fno-omit-frame-pointer -fno-stack-protector` means retaining the frame-pointer and disabling stack protection.

- `-Wall -Wno-unused-variable -Werror -ggdb :`

Enables all warnings except for unused variables, treats warnings as errors, and includes GDB debugging info.

- `-Ios -std=gnu17 -O2 -c os/entry.S -o build/os/entry.o :`

Uses the `os` directory for includes, adheres to the GNU17 C standard, enables O2 optimization

compiles (`-c`) `os/entry.S` into `build/os/entry.o`.

The linker (`ld`) combines all `.o` files into the final kernel ELF file.

```
riscv64-unknown-elf-ld -z max-page-size=4096 -T os/kernel.ld -o build/kernel build/os/
console.o build/os/main.o build/os/printf.o build/os/sbi.o build/os/string.o build/os/
entry.o
```

following series of commands indicates:

- `-T os/kernel.ld`: Specifies `os/kernel.ld` as the linker script.
- `-o build/kernel`: Outputs to `build/kernel`.
- Inputs are all `.o` files.

LINKER SCRIPT

The `kernel.ld` linker script dictates how the linker arranges sections from the `.o` files (Memory Layout) and how the resulting ELF file is loaded into memory (Program Headers).

Memory Layout

Memory Layout refers to the organization and structure of a program's data and code in memory during execution.

Here's the content of `kernel.ld`:

```
OUTPUT_ARCH(riscv)
ENTRY(_entry)
BASE_ADDRESS = 0x80200000;

SECTIONS
{
    . = BASE_ADDRESS;
    skernel = .;

    s_text = .;
    .text : {
        *(.text.entry)
        *(.text .text.*)
        . = ALIGN(4K);
        e_text = .;
    }

    . = ALIGN(4K);
    s_rodata = .;
    .rodata : {
        *(.rodata .rodata.*)
    }

    . = ALIGN(4K);
    e_rodata = .;
    s_data = .;
    .data : {
        *(.data.apps)
```

```

        *(.data .data.*)
    }

    . = ALIGN(4K);
    e_data = .;
    .bss : {
        *(.bss.stack)
        s_bss = .;
        *(.bss .bss.*)
    }

    . = ALIGN(4K);
    e_bss = .;
    ekernel = .;

    /DISCARD/ : {
        *(.eh_frame)
    }
}

```

The `OUTPUT_ARCH` at the beginning specifies that the generated ELF file is compatible with the RISC-V architecture, and `ENTRY` indicates that the entry point of the ELF file is the symbol `_entry`. Then, we define a constant `BASE_ADDRESS` and set it equal to `0x80200000`, which is also the starting address of our kernel.

In the `SECTIONS` block, we first define the current address (represented by the `.` symbol) (imagine the linker is assigning locations to all sections starting from a certain address) as `0x80200000`, and export the values of the symbols `skernel` and `s_text` as the current address.

Subsequently, we define the `.text` section, which typically represents the code segment:

```

.text : {
    *(.text.entry)
    *(.text .text.*)
    . = ALIGN(4K);
    e_text = .;
}

```

Places `.text.entry` first (defined in `entry.S`), followed by other `.text` sections.

The `.text.entry` section is defined in `entry.S`:

```

.section .text.entry
.globl _entry
_entry:

```

By placing `_entry` in `.text.entry` and prioritizing it in the linker script, **we ensure `_entry` is at the kernel's starting address, `0x80200000`, where execution begins.**

Verify this via disassembly:

```
$ llvm-objdump-19 -d build/kernel | less
build/kernel:  file format elf64-littleriscv

Disassembly of section .text:

0000000008020000 <skernel>:
80200000: 00007117    auipc    sp, 0x7
80200004: 00010113    mv       sp, sp
80200008: 00000097    auipc    ra, 0x0
8020000c: 174080e7    jalr     0x174(ra) <main>
```

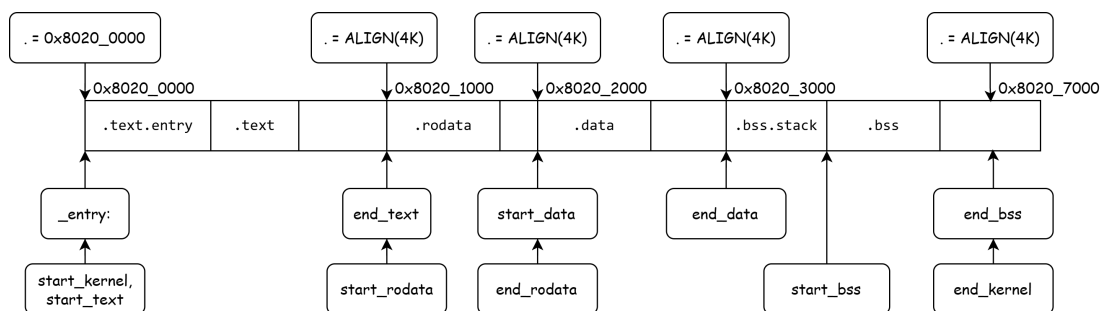
Then, we align the current address to a 4K boundary and export the `e_text` symbol.

Remaining sections:

- `.rodata`: Read-only data.
- `.data`: Read-write data.
- `.bss`: Data initialized to zero at startup.

All data sections are non-executable.

Here's a diagram of the kernel image's memory layout:



Finally, inspect the ELF file with `readelf`:

```
$ /d/o/SUSTech-OS-2025 (xv6-lab1)> llvm-readelf-19 -a build/kernel
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                           ELF64
  Data:                               2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                        0
  Type:                               EXEC (Executable file)
  Machine:                            RISC-V
  Version:                            0x1
  Entry point address:                0x80200000
  Start of program headers:           64 (bytes into file)
  Start of section headers:          29208 (bytes into file)
  Flags:                              0x4, double-float ABI
  Size of this header:                64 (bytes)
```

```

Size of program headers:      56 (bytes)
Number of program headers:    4
Size of section headers:     64 (bytes)
Number of section headers:    19
Section header string table index: 18

```

There are 19 section headers, starting at offset 0x7218:

Section Headers:

[Nr]	Name	Type	Address	Off	Size	ES	Flg	Lk	Inf	Al
[0]		NULL	0000000000000000	000000	000000	00		0	0	0
[1]	.text	PROGBITS	0000000080200000	001000	001000	00	AX	0	0	4
[2]	.rodata	PROGBITS	0000000080201000	002000	0001f0	00	A	0	0	8
[3]	.data	PROGBITS	0000000080202000	003000	000008	00	WA	0	0	8
[4]	.bss	NOBITS	0000000080203000	003008	004000	00	WA	0	0	1
[5]	.debug_info	PROGBITS	0000000000000000	003008	0010c0	00		0	0	1
[6]	.debug_abbrev	PROGBITS	0000000000000000	0040c8	0006e5	00		0	0	1
[7]	.debug_loclists	PROGBITS	0000000000000000	0047ad	0004b5	00		0	0	1
[8]	.debug_aranges	PROGBITS	0000000000000000	004c70	000130	00		0	0	16
[9]	.debug_line	PROGBITS	0000000000000000	004da0	000fe7	00		0	0	1
[10]	.debug_str	PROGBITS	0000000000000000	005d87	0003bc	01	MS	0	0	1
[11]	.debug_line_str	PROGBITS	0000000000000000	006143	0000d3	01	MS	0	0	1
[12]	.comment	PROGBITS	0000000000000000	006216	00001a	01	MS	0	0	1
[13]	.riscv.attributes	RISCV_ATTRIBUTES	0000000000000000	006230	000065	00		0	0	1
[14]	.debug_frame	PROGBITS	0000000000000000	006298	0004c8	00		0	0	8
[15]	.debug_rnglists	PROGBITS	0000000000000000	006760	00007f	00		0	0	1
[16]	.symtab	SYMTAB	0000000000000000	0067e0	0006f0	18		17	31	8
[17]	.strtab	STRTAB	0000000000000000	006ed0	000279	00		0	0	1
[18]	.shstrtab	STRTAB	0000000000000000	007149	0000cc	00		0	0	1

Key to Flags:

W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
 L (link order), O (extra OS processing required), G (group), T (TLS),
 C (compressed), x (unknown), o (OS specific), E (exclude),
 R (retain), p (processor specific)

Elf file type is EXEC (Executable file)

Entry point 0x80200000

There are 4 program headers, starting at offset 64

Program Headers:

Type	Offset	VirtAddr	PhysAddr	FileSiz	MemSiz	Flg	Align
ATTRIBUTES	0x006230	0x0000000000000000	0x0000000000000000	0x000065	0x000000	R	0x1
LOAD	0x001000	0x0000000080200000	0x0000000080200000	0x0011f0	0x0011f0	R E	0x1000
LOAD	0x003000	0x0000000080202000	0x0000000080202000	0x000008	0x005000	RW	0x1000
GNU_STACK	0x000000	0x0000000000000000	0x0000000000000000	0x000000	0x000000	RW	0x10

Section to Segment mapping:

Segment Sections...

```

00 .riscv.attributes
01 .text .rodata
02 .data .bss
03

```

```
None .debug_info .debug_abbrev .debug_loclists .debug_aranges .debug_line .debug_str .debug_line_str .comm
```

- The final file type is: `Type: EXEC (Executable file)`
- The entry point address is: `Entry point address: 0x80200000`
- There are a total of 19 Sections:
 - Each Section has its own Flags, where `A` indicates that this Section should be allocated memory space during loading, `w` indicates it is writable, and `x` indicates it is executable.
- There are a total of 4 Program Headers, two of which are LOAD:
 - The first LOAD indicates:
 - At virtual address (VirtAddr) `0x80200000`, it maps to physical address (PhysAddr) `0x80200000`, allocating a memory space of `0x0011d0` bytes (MemSiz). This memory segment has the permission `RE` (Read & Executable).
 - It copies `0x0011d0` bytes (FileSiz) from the ELF file's offset `0x001000` to the above memory space.
 - The second LOAD indicates:
 - At virtual address `0x80202000`, it maps to physical address `0x80202000`, allocating a memory space of `0x005000` bytes. This memory segment has the permission `RW` (Read & Write).
 - It copies `0x0008` bytes from the ELF file's offset `0x3000` to this memory segment. However, the memory segment allocates `0x5000` bytes of space, **indicating that the remaining unfilled space is entirely zero, which is the bss segment.**
- In the `Section to Segment mapping:` section, we can see:
 - The first Program Header includes the `.text` and `.rodata` Sections.
 - The second Program Header includes the `.data` and `.bss` segments.

Continue Kernel Execution

The first few instructions executed:

```
80200000: 00007117    auipc    sp, 0x7
80200004: 00010113    mv       sp, sp
80200008: 00000097    auipc    ra, 0x0
8020000c: 174080e7    jalr     0x174(ra) <main>
```

Source code (`entry.s`):

```
.section .text.entry
.globl _entry
_entry:
    lla sp, boot_stack_top
    call main

.section .bss.stack
.globl boot_stack
boot_stack:
    .space 4096 * 4
    .globl boot_stack_top
boot_stack_top:
```

In this segment of code:

- We utilize the `auipc` and `addi` instructions to direct the stack pointer towards `boot_stack_top`, a stack we have pre-allocated for the initial kernel entry.
- Through the `auipc` and `jalr` instructions, we execute a jump to the main function to continue the program's execution.

Why Use Assembly for Kernel Entry?

At the kernel entry, OpenSBI does not set the stack pointer (`sp`). Since C requires a stack for local variables and function calls, we use assembly to initialize `sp`.

The kernel then jumps to `main` in `main.c`:

observe main function in file `main.c`

```
void main() {
    printf("\n");
    printf("clean bss: %p - %p\n", s_bss, e_bss);
    memset(s_bss, 0, e_bss - s_bss);
    printf("Kernel booted.\n");
    printf("Hello World!\n");

    printf("sysregs:\n");
    print_sysregs();
    printf("%d\n", global_variable);

    printf("kernel ends, parking...\n");
}
```

```
while (1);
}
```

- Clears the `.bss` section (normally done by the Linux kernel, but we're the kernel now).
- Prints messages and enters an infinite loop.

```
clean bss: 0x0000000080207000 - 0x0000000080207000
Kernel booted.
Hello World!
```

? Lab Report 3

Attempt to read the `mvendorid` CSR.

In `main.c`, add before the `while(1);` loop:

```
uint64 x;
asm volatile("csrr %0, mvendorid : "=r"(x));
printf("csr: %p\n", x);
```

Compile and run with `make run`. Can your program read the `mvendorid` CSR value?

Refer to the RISC-V Privileged Manual ([riscv-privileged.pdf](#)), Sections 2.1 (CSR Address Mapping Conventions) and 2.2 (CSR Listing), and explain why your program might restart infinitely.

SBI CALL

In this lab's kernel, `printf` calls `consputc` to print characters to the console, which ultimately invokes `sbi_call(SBI_CONSOLE_PUTCHAR, c, 0, 0)`.

To access hardware functions (e.g., `sbi_console_putchar` and `sbi_console_getchar`), we set registers like `a0` and `a1`, then use the `ecall` instruction to request service from OpenSBI in **M mode**. OpenSBI processes the request and returns.

i Inline Assembly

ecall (environment call): When we execute this instruction in S-mode, it triggers an `ecall-from-s-mode-exception`, entering the interrupt handling process in M-mode (e.g., setting a timer, etc.); when we execute this instruction in U-mode, it triggers an `ecall-from-u-mode-exception`, entering the interrupt handling process in S-mode (commonly used for system calls).

Note that C language cannot directly invoke `ecall`; it needs to be implemented using inline assembly.

i Calling Convention

A **Calling Convention** defines the rules for function calls between the caller and callee.

including:

1. How parameters and return values are passed.
2. Which registers the caller and callee must preserve.

SBI's calling convention (riscv-sbi.pdf):

All SBI functions share a single binary encoding, which facilitates the mixing of SBI extensions. The SBI specification follows the below calling convention.

- An ECALL is used as the control transfer instruction between the supervisor and the SEE.
- a7 encodes the SBI extension ID (EID),
- a6 encodes the SBI function ID (FID) for a given extension ID encoded in a7 for any SBI extension defined in or after SBI v0.2.
- All registers except a0 & a1 must be preserved across an SBI call by the callee.
- SBI functions must return a pair of values in a0 and a1, with a0 returning an error code.

For legacy extensions (e.g., `putchar` / `getchar`):

- Nothing is returned in a1 register.
- All registers except a0 must be preserved across an SBI call by the callee.
- The value returned in a0 register is SBI legacy extension specific.

According to the RISC-V Calling Convention, parameters are passed through the a series of registers, starting from `a0`.

In the `sbi_call_legacy` function, we first declare four variables and specify that they should be directly assigned to the corresponding registers (`register`, `asm("a0")`).

Next, we assign the value of `a7` to indicate which SBI function we want to call, and assign the three parameters to `a0`, `a1`, and `a2` respectively.

Then, we use `asm volatile` to perform the `ecall` instruction.

Finally, we return the value of the `a0` register.

```
const uint64 SBI_CONSOLE_PUTCHAR = 1;
const uint64 SBI_CONSOLE_GETCHAR = 2;

static int inline sbi_call_legacy(uint64 which, uint64 arg0, uint64 arg1, uint64 arg2)
{
    register uint64 a0 asm("a0") = arg0;
    register uint64 a1 asm("a1") = arg1;
    register uint64 a2 asm("a2") = arg2;
    register uint64 a7 asm("a7") = which;
    asm volatile("ecall" : "=r"(a0) : "r"(a0), "r"(a1), "r"(a2), "r"(a7) : "memory");
    return a0;
}

void console_putchar(int c) {
    sbi_call_legacy(SBI_CONSOLE_PUTCHAR, c, 0, 0);
}
```

`asm volatile` is used to directly insert assembly code (inline assembly) into C code. It consists of four parts:

```
asm asm-qualifiers ( AssemblerTemplate
                    : OutputOperands
                    [ : InputOperands
                    [ : Clobbers ] ])
```

We use `volatile` as an `asm-qualifier` to indicate that the compiler should not perform any potential optimizations on this segment of code.

`AssemblerTemplate` represents the template of the assembly code. `OutputOperands` specifies which C variables should be treated as the output of this assembly segment. `InputOperands` specifies which C variables should be treated as the input of this assembly segment. `Clobbers` indicates which registers or system states will be modified by this assembly code.

- See: <https://gcc.gnu.org/onlinedocs/gcc-12.1.0/gcc/Extended-Asm.html>

? Lab Report 4

Call the newer SBI function `sbi_get_spec_version()`.

In `main.c`, add before the `while(1);` loop:

```
struct sbiret ret = sbi_call(?, ?, 0, 0, 0);
printf("sbicall: err: %d, ret: %p\n", ret.error, ret.value);
```

Refer to the SBI manual (`riscv-sbi.pdf`) to fill in the `?` values. Compile and run with `make run`, then explain the output values.